

FUNCTION

- **DEFINITION:-** A function is a subprogram that acts on data and often returns a value.

The most important reason to use function is to make program handling easier as only a small part of the program is dealt at a time.

Each function has its own name. When that name in a program, that function is executed at that time and when the execution is over, execution returns back to calling code.

- **TYPES OF FUNCTION**

In C++ there are 2 types of functions:

1. Built In or Library functions
2. User-Defined Functions

Built In functions:- These functions are part of compiler packages. These are part of standard library made available by the compiler. Eg- `exit()`, `sqrt()`, `pow()`, `strlen()` are few library functions or built in functions.

User Defined functions:- The user defined functions are created by programmer itself. These functions are created as per requirements of program.

- **FUNCTION DEFINITION**

The general form of function definition is as given below:

```
type function_name(parameter list)
{
    Body of the function
}
```

In above syntax, type specifies the type of value that the return statement of the function returns. It may be any valid C++ data types. If no type is specified the compiler assumes that the function returns an integer value. The parameter list is a comma separated list of variables of a function referred to as its arguments. A function may be without any parameters, where the parameters list is empty.

- **FUNCTION PROTOTYPE**

A function prototype is a declaration of the function that tells the program about the type of the value returned by the function and the number and type of arguments.

A declaration introduces a function name to the program whereas a definition is a declaration that also tells the program what the function is doing and how it is doing.

Function prototype enables a compiler to carefully compare each use of the function with the prototype to determine whether the function is invoked properly i.e. the number and types of arguments are compared and any wrong number or type of the argument is reported.

General form of function prototype :

Type `function_name(parameter list);`

- **FOUR FUNCTION TYPE**

TYPE 1: *void function with NO arguments*

This type of function simply performs an independent task. It DOES NOT send or receive any parameters and it DOES NOT RETURN any value.

```
void function_name()
{
    Body of function
}
```

```

example:-void area( )           //function definition
{
    float a,r =10.66;
    a=3.14*r*r;
    cout<<"\n area of circle"<<a;
}
void main()
{
    area();           //call to function area()
    getch();
}

```

TYPE 2: void function with SOME arguments

This type of function DOES receive some parameters but DOES NOT RETURN any value.

```

void function_name( arguments list)
{
    Body of function
}

```

```

example:-void area( float r )       //function definition
{
    float a;;
    a=3.14*r*r;
    cout<<"\n area of circle"<<a;
}
void main()
{
    float r=10.67;
    area( r);           //call to function area()
    getch();
}

```

TYPE 3: *NON-void function with NO arguments*

This type of function takes NO arguments but DOES RETURN any value.

```
return-type function_name( )  
{  
    body of function  
    return(value);  
}
```

```
example:-float area( )      //function definition  
{  
    float a,r=18.9;;  
    a=3.14*r*r;  
    return(a);  
}  
void main()  
{  
    float a;  
    a= area( );              //call to function area()  
    cout<<"\n area of circle"<<a;  
    getch();  
}
```

TYPE 4: *NON-void function with SOME arguments*

This type of function takes *SOME* arguments and *DOES RETURN* any value.

```
return-type function_name( arguments list )  
{  
    body of function  
    return(value);  
}
```

```

example:-float area( float r )           //function definition
{
    float a,
    a=3.14*r*r;
    return(a);
}

void main()
{
    float a,r=19,78
    a= area( r);           //call to function area()
    cout<<"\n area of circle"<<a;
    getch();
}

```

Call By Value & Call by References

A function can be invoked in 2 ways:

- 1.Call by Value
- 2.Call by Reference

Call by Value

The call by value method copies the values of **actual parameters** (the parameter that appear in a function call statement are actual parameters) to formal parameter (the parameter that appear in function definition are formal parameters)

The main advantage of call by value method is that programmer cannot change the variables that are used to call the function because any change that occurs inside function is on the function's copy of the argument value.The original copy of the argument value remains intact.

EXAMPLE :- Write a program in C++ to swap two numbers using function.

```

#include<iostream.h>
#include<conio.h>

```

```

void swap(int a,int b);           //function prototype
int main( )
{
    clrscr( );
    int a=5,b=7;
    cout<<"\n original numbers\n a= "<<a<<" \t b= "<<b;
    swap(a,b)                     //call to a function
    cout<<"\n swapped numbers \n a="<<a<<"\t b= "<<b;
    getch( );
    return 0;
}
void swap(int a,int b)           //function definition
{
    int temp;
    temp=a;
    a=b;
    b=temp;
}

```

In above example when function swap() is invoked ,the values of a and b(actual parameters) get copied on to x and y (formal parameters).The swap() does swao the values but x and y values are swapped ,a and b values ,nothing happens to them.

Example

```

void main( )
{
    int a=10,ans;
    int showchange(int);
    cout<<"\n The original value= "<<a;
    ans=showchange(a);
    cout<<"\n Return value of function = "<<ans;
    cout<<"\n The value after function showchange( ) is executed=
"<<a;
    getch( );
}

```

```

}
int showchange(int x)
{
    x=20;
    return (x);
}

```

OUTPUT

The original value= 10

Return value of function = 20

The value after function showchange() is executed= 10

In the above example, the value of argument to showchange() i.e **a**.

10 is copied into parameters **x** so that **x** gets value 10.

When the statement **x=20** takes place the value of **x** is changed but not the value of **a**. The **a** has still the value **10**.

Call by Reference

In call by reference method in place of a value to the function being called, a reference to the original variable is passed. Reference is a different name for predefined variable. i.e the same variable value can be accessed by any of the 2 names i. e the original variable's name and the reference name.

Eg. In housing society, eg Mr. Ram Kapoor. We can refer to Mr. Ram Kapoor either by his First name or by his original name Mr. Ram Kapoor. Both the names refer to the same person.

Provisions of the reference variables in C++ permits us to pass parameters to the function by references.

In call by references method, the called function does not create its own copy of original values rather, it refers to the original values only by different names

i.e the reference .Thus the called function works with the original data and any changes in the values gets reflected to the data.

The call by reference method is useful in situation where the values of the original variables are to be changed using a function.

EXAMPLE :- Write a program in C++ to swap two numbers using call by reference.

```
#include<iostream.h>
#include<conio.h>
void swap(int &x, int &y);           //function prototype
int main( )
{
    clrscr( );
    int a=5,b=7;
    cout<<"\n original numbers\n a= "<<a<<" \t b= "<<b;
    swap(a,b)                       //call to a function
    cout<<"\n swapped numbers \n a="<<a<<"\t b= "<<b;
    getch( );
    return 0;
}
void swap(int &x,int &y)             //function definition
{
    int temp;
    temp=x;
    x=y;
    y=temp;
    cout<<"\n Swapped values are  a="<<x <<" b= "<<y;
}
}
```

In above example the values of a and b have been swapped because this time swap() works with original data.

“Return” Statement

The keyword return is used to terminate execution of the function and return control back to the statement where the call to the function was made i.e calling function.It can also be used to return a single value to the calling function. Keyword return is used to terminate execution of the function.

Syntax:

return;
 or
 return expression;
 or
 return (expression);

- **Difference between Call by Value and Call by Reference**

Sr.No	Call by Value	Call by Reference
1.	The value of actual argument is copied into formal arguments	The formal argument represents the actual argument.
2.	The function cannot access the original variable	The function can access the original variable
3.	Any changes made to the formal argument are not made to the actual argument.	Any changes made to the formal argument are reflected to the actual argument
4.	No need to use Pointers	Requires pointers to be used
5	Used when we only need the value of a variable but don't need to modify it.	Used when a variable has to be modified by the function.

PROGRAMS

1. Write a C++ program to find the smallest of four given integers using min() function to return the smallest of four given integers .
 int min(int,int,int,int) *March 2013*

Solution:

```
#include<iostream.h>
int min(int n1,int n2,int n3,int n4)

void main( )
{
    int a,b,c,d,small;
    cout<<"\n enter the 4 nos";
    cin>>a>>b>>c>>d;
    small=min(a,b,c,d);
    cout<<"\n the smallest number is = "<< small;
    getch( );
}
```

```

}

int min(int n1,int n2,int n3,int n4)
{
    int s;
    if(n1<n2)
        s=n1;
    else
        s=n2;
    if(n3<s)
        s=n3;
    if(n4<s)
        s=n4;
    return(s);
}

```

2. Write a c++ program with computetriangle() function that displays the area a and perimeter p of a triangle with given side lengths x,y and z,
void computetriangle(float &x, float &y,float &z ,float &a,float &p);
OCT 2007

Solution:

```

#include<iostream.h>
#include<math.h>
void computetriangle( float &x, float &y,float &z,float &a,float &p);
void main( )
{
    float x,y,z,a,p;
    cout<<"\n enter three side values of triangle";
    cin>>x>>y>>z;
    computetraingle(x,y,z,a,p);
    getch( );
}

void computetriangle( float &x, float &y,float &z, float &a,float &p)
{
    flaot s;
    p=x + y+ z;
    cout<<"\n Perimeter of triangle = "<<p;

```

```

s=p/2.0;
a=sqrt(s * (s-x) * (s- y) *( s- z));
cout<<"\n Area of triangle ="<<a;
}

```

3. Write a program in C++ to accept 2 integers number in main() and find sum of those numbers using function by address.

OCT 2015

Solution:

```

int add(int *,int *);           //function prototype
void main( )
{
    int a,b,sum;
    cout<<"\n enter 2 nos",
    cin>>a>>b;
    sum=add(&a,&b);             //call to function
    cout<<"\n sum of 2 nos = " <<sum;
    getch( );
}

int add(int  *x, int  *y)       //function definition
{
    int s;
    s=*x+*y;
    return (s);
}

```

4. Write a program in C++ to accept 2 integers number in main() ,pass them to function great() using call by value and find greater number function great() should not return any value.

March 2015

Solution:

```

void great(int, int );         //function prototype
void main( )
{
    int a,b;
    cout<<"\n enter 2 nos",
    cin>>a>>b;
    great(a,b);                //call to function
}

```

```

        getch( );
    }

Void great(int x, int y)           //function definition
{
    If(x > y )
        Cout<<"\n The greatest number is"<< x;
    Else
        Cout<<"\n The greatest number is"<< y;
}

```

5. Write a C++ program with sphere() function that returns the volume V and the surface s of a sphere with given radius r.
 void sphere(double &a, double &v, double r); OCT 2013,
MAR 2013

Solution:

```

#include<iostream.h>
void sphere( double &a, double &v, double r);
void main( )
{
    double a,v,r;
    cout<<"\nEnter radius";
    cin>>r;
    sphere(a ,v ,r);
    cout<<"\n The area with radius "<< r <<" is " << a <<"and its volume
"<<v;
    getch( );
}

```

```

void sphere( double &a, double &v, double r)
{
    a =4.0 * 3.14 * r;
    v= a * r /3.0 ;
}

```

6. Write a C++ program to find the largest of four given integers using max() function to return the smallest of four given integers .
 int max(int,int,int,int) March 2013

Solution:

```
#include<iostream.h>
int max(int n1,int n2,int n3,int n4)

void main( )
{
    int a,b,c,d,big;
    cout<<"\n enter the 4 nos";
    cin>>a>>b>>c>>d;
    big=max(a,b,c,d);
    cout<<"\n the largest number is = "<< big;
    getch( );
}

int max(int n1,int n2,int n3,int n4)
{
    int b;
    if(n1 > n2)
        b=n1;
    else
        b=n2;
    if(n3 > b )
        b=n3;
    if(n4 > b)
        b=n4;
    return(b);
}
```

7. . Write a C++ program with `computecircle( )` function that returns the area `a` and the circumference `c` of a circle with given radius `r`.

`void computecircle( float &a, float &c, float r);`
`OCT 2010`

`OCT 2007,`

```
#include<iostream.h>
void computecircle( float &a, float &c, float r);
void main( )
```

```

{
    float a,c,r;
    cout<<"\nEnter radius";
    cin>>r;
    computecircle(a ,c ,r);
    cout<<"\n The area of Circle  with radius  "<< r <<" is " << a <<"and
its Circumference "<<c;
    getch( );
}

```

```

void computecircle( float &a, float  &c, float r)
{
    a = 3.14 * r*r;
    c = 2 * 3 .14 * r;
}

```